# unique_ptr — Խորը ուղեցույց

### C++ Smart Pointer presentation

Այս ուղեցույցը մանրամասն բացatrum e `std::unique_ptr`-y՝ inch e, inchpes e ashxatum, ownership, move semantics, make_unique, custom deleter, orinaknerov ev interview harcerov.

---

# 1. Ի՞նչ է unique_ptr-ը

`std::unique_ptr`-y smart pointer e, vor **exclusive ownership** uni resource-i vra. Mek resource-y karox e patkanel **miayn mek** unique_ptr-i. Scope-ic durs galis `avtomat` delete`.

```
#include <memory>

std::unique_ptr<int> p = std::make_unique<int>(10);
cout << *p;         // 10
// delete petq che -- scope verjum avtomat
```

RAII-i lavaguyn orinak` heap memory-n karavarvum e avtomat, chka leak.

---

# 2. Exclusive Ownership — copy CHKA

```
std::unique_ptr<int> a = std::make_unique<int>(5);
std::unique_ptr<int> b = a;              // COMPILE ERROR -- copy chka
std::unique_ptr<int> c = std::move(a);   // OK -- ownership poxancvum
// hima a == nullptr, c-n uni resource-y
```

unique_ptr-y **chi copy linum** (miayn mek owner): Ownership-y poxanceluc `std::move` -> hin pointer-y darnum nullptr.

---

# 3. make_unique (naxentreli)

```
auto p = std::make_unique<int>(10);           // int
auto arr = std::make_unique<int[]>(100);      // array
auto obj = std::make_unique<MyClass>(a, b);   // object ctor args-ov
```

Inchu make_unique (ne `unique_ptr<int>(new int)`). - Aveli karch, exception-safe - Chka baciahayt `new`

---

# 4. Karevor metodner

```
auto p = std::make_unique<int>(10);

*p;              // dereference
p.get();         // raw pointer (ownership chi poxancum)
p.reset();       // azatum resource-y (delete), p = nullptr
p.reset(new int(20));  // hin azatum, nor vercnum
p.release();     // ownership hanum, veradarcnum raw pointer (delete CHI anum)
if (p) { }       // stugum` null che
```

---

# 5. Ownership transfer (function-neri het)

```
std::unique_ptr<int> create() {
    return std::make_unique<int>(42);   // ownership veradarcnum (move avtomat)
}

void consume(std::unique_ptr<int> p) {  // ownership vercnum
    cout << *p;
}   // p ochncvum -> delete

auto p = create();
consume(std::move(p));   // ownership poxancum -> hima p == nullptr
```

---

# 6. Custom Deleter

```
auto fileDeleter = [](FILE* f) { if (f) fclose(f); };
std::unique_ptr<FILE, decltype(fileDeleter)> file(fopen("data.txt","r"), fileDeleter);
// scope verjum -> fclose avtomat
```

Custom deleter-ov karas karavarel voch-memory resource (file, socket, handle).

---

# 7. Inchu unique_ptr

```
- Zero overhead (raw pointer-i chapov, arag)
- Exclusive ownership -> paŕz seperutyan model
- Avtomat delete -> chka leak
- Move-only -> chka patahakan copy
- DEFAULT smart pointer (ete shared petq che)
```

---

# 8. unique_ptr vs raw pointer

```
int* raw = new int(5);   // dzeŕqov delete petq, leak-i vtang
// vs
auto smart = std::make_unique<int>(5);   // avtomat, safe
```

---

# 9. Interview harcer & patasxanner

**unique_ptr inch e?**

> *Smart pointer exclusive ownership-ov; mek resource -> mek owner; scope verjum avtomat delete.*

**Inchu copy chka?**

> *Exclusive ownership -- erku owner -> double delete. Miayn move (ownership transfer).*

**make_unique vs unique_ptr(new)?**

> *make_unique aveli karch, exception-safe, chka baciahayt new.*

**Ownership inchpes es poxancum?**

> *std::move -> hin pointer nullptr darnum.*

**get() vs release()?**

> *get() raw pointer talis (ownership pahum); release() ownership hanum, raw talis (delete chi anum).*

**reset() inch e anum?**

> *Azatum e nerkayis resource-y (delete), optional nory vercnum.*

**Overhead?**

> *Grethe zero -- raw pointer-i chapov, arag.*

---

# Cheat Sheet

```
unique_ptr = EXCLUSIVE ownership (mek owner), avtomat delete, zero overhead

  make_unique<T>(args)   -> steghcum (NAXENTRELI)
  *p / p->               -> dereference
  p.get()               -> raw pointer (ownership pahum)
  p.release()           -> ownership hanum (delete CHI)
  p.reset()             -> azatum (delete)
  std::move(p)          -> ownership poxancum (p -> nullptr)

  COPY CHKA (miayn move) | move-only
  DEFAULT smart pointer (shared petq chlinelu depqum)
```